

# Ierarhii de clase

Şase teme OOP — de rezolvat doar cu moştenire şi polimorfism · Java · Adrian

**Ce ai voie să foloseşti:** clase, câmpuri `private / protected`, `getter` şi `setter`i, mai mulţi constructori pe aceeaşi clasă, `extends`, `super(...)` şi `super.metoda()`, `@Override`, `instanceof`, `cast`, `List + ArrayList`, `for-each`, `if/else`, `System.out.println`.

**Încă NU folosim:** `abstract`, `interface`, `toString()`, `equals / hashCode`, `clone()`, `static`, fişiere, `try/catch`, `stream-uri`, interfaţă grafică. Totul se „afişează” cu `System.out.println`.

**Cum se lucrează:** câte un pachet nou pentru fiecare temă. Ierarhia o decizi tu — ce clasă e superclasă, ce este subclasă, ce câmpuri urcă şi ce metode se suprascriu. Nu există o singură soluţie corectă; există soluţii în care **adăugarea unui element nou nu te obligă să modifice codul existent**, şi soluţii în care te obligă.

## Partea 1 — la nivelul temei de la clasă

### 1. Orarul unei zile

nivel 1

Să se definească o ierarhie de clase pentru un modul care are ca scop **afişarea** şi **manipularea** unui **orar**. Un orar este format din **zero sau mai multe intervale**, din setul:

| Element           | Definire                                                          |
|-------------------|-------------------------------------------------------------------|
| Oră de curs       | Ora de început, durata în minute, materia, sala                   |
| Pauză             | Ora de început, durata în minute                                  |
| Activitate liberă | Ora de început, durata în minute, denumirea, locul de desfăşurare |

**Manipulare** înseamnă:

- **Decalarea** întregului orar cu un număr de minute dat (toate intervalele se mută cu aceeaşi valoare);
- **Duplicarea** orarului — un orar nou, cu aceleaşi intervale la aceleaşi ore.

**Cerinţe:** defineşte toate clasele necesare şi scrie un program care creează, afişează şi manipulează două ore.

## 2. Bonul de casă

nivel 1

Să se definească o ierarhie de clase pentru un modul care are ca scop **afișarea** și **manipularea** unui **bon de casă**. Un bon este format din **zero sau mai multe poziții**, din setul:

| Element            | Definire                                           |
|--------------------|----------------------------------------------------|
| Prodot la bucată   | Denumirea, prețul unei bucăți, numărul de bucăți   |
| Prodot la kilogram | Denumirea, prețul pe kilogram, cantitatea în grame |
| Ambalaj returnabil | Denumirea, garanția unei bucăți, numărul de bucăți |

Fiecare poziție are o **valoare de plată**, calculată diferit în funcție de tip. Bonul are un **total**.

**Manipulare** înseamnă:

- **Scumpirea** întregului bon cu un procent dat;
- **Duplicarea** bonului.

**Cerințe:** definește toate clasele necesare și scrie un program care creează un bon cu cel puțin cinci poziții, îl afișează, îl scumpește cu 10% și afișează din nou totalul.

## Partea 2 — un pic mai grele

## 3. Expedierea de colete

nivel 2

Să se definească o ierarhie de clase pentru un modul care are ca scop **afișarea** și **manipularea** unei **expedieri**. O expediere este formată din **zero sau mai multe colete**, din setul:

| Element | Definire                                                                         |
|---------|----------------------------------------------------------------------------------|
| Plic    | Destinatarul, greutatea în grame                                                 |
| Cutie   | Destinatarul, greutatea în grame, lungimea, lățimea, înălțimea                   |
| Palet   | Destinatarul și <b>coletele așezate pe el</b> (zero sau mai multe, de orice tip) |

Fiecare colet are o **greutate totală** (pentru palet: greutatea lui plus a tot ce e pe el) și un **cost de transport**, calculat diferit pentru fiecare tip.

**Manipulare** înseamnă:

- **Redirecționarea** întregii expedieri către alt destinatar;
- **Duplicarea** expedierii.

**Cerințe:** definește toate clasele necesare și scrie un program care creează o expediere ce conține și un palet cu colete pe el, o afișează, o redirecționează și afișează greutatea totală.

#### 4. Casa inteligentă

nivel 2

Să se definească o ierarhie de clase pentru un modul care are ca scop **afișarea** și **manipularea** unei **locuințe**. O locuință are **zero sau mai multe dispozitive**, din setul:

| Element           | Definire                                                  |
|-------------------|-----------------------------------------------------------|
| Bec               | Camera, starea (aprins / stins), intensitatea în procente |
| Termostat         | Camera, temperatura setată                                |
| Senzor de mișcare | Camera, starea (armat / dezarmat)                         |
| Priză inteligentă | Camera, starea (pornită / oprită), consumul în wați       |

**Manipulare** înseamnă:

- **Plecarea de acasă** — o singură comandă dată locuinței, care are efect **diferit** pe fiecare tip de dispozitiv (becurile se sting, termostatele coboară cu 5 grade, senzorii se armează, prizele se opresc);
- **Întoarcerea acasă** — comanda inversă;
- **Duplicarea** locuinței — aceeași configurație, pentru alt etaj.

**Cerințe:** definește toate clasele necesare și scrie un program care afișează starea locuinței înainte și după plecarea de acasă. Codul care dă comanda **nu are voie să întrebe ce tip de dispozitiv este**.

### Partea 3 — provocări

#### 5. Documentul formatat

provocare

Să se definească o ierarhie de clase pentru un modul care are ca scop **afișarea** și **manipularea** unui **document**. Un document este format din **zero sau mai multe blocuri**, din setul:

| Element  | Definire                                                                                                  |
|----------|-----------------------------------------------------------------------------------------------------------|
| Paragraf | Textul                                                                                                    |
| Titlu    | Textul și nivelul (1, 2 sau 3)                                                                            |
| Citat    | Textul și autorul                                                                                         |
| Listă    | Elementele listei (zero sau mai multe texte)                                                              |
| Secțiune | Un titlu și <b>blocurile din interiorul ei</b> (zero sau mai multe, de orice tip, inclusiv alte secțiuni) |

**Afișarea** se face indentat: fiecare bloc dintr-o secțiune se scrie deplasat la dreapta cu două spații față de secțiunea care îl conține.

**Manipulare** înseamnă:

- **Coborârea cu un nivel** a întregului document — toate titlurile trec de la nivelul  $n$  la  $n+1$ ; decide și scrie în comentariu ce se întâmplă cu un titlu care e deja pe nivelul 3;
- **Duplicarea** documentului.

**Cerințe:** definește toate clasele necesare și scrie un program care construiește un document cu cel puțin o secțiune în altă secțiune, îl afișează, îl coboară cu un nivel și îl afișează din nou.

## 6. Formularul de înscriere

provocare

Să se definească o ierarhie de clase pentru un modul care are ca scop **afișarea** și **manipularea** unui **formular**. Un formular este format din **zero sau mai multe câmpuri**, din setul:

| Element                 | Definire                                                            |
|-------------------------|---------------------------------------------------------------------|
| <b>Câmp text</b>        | Eticheta, valoarea completată, lungimea maximă admisă               |
| <b>Câmp numeric</b>     | Eticheta, valoarea completată, valoarea minimă și cea maximă admise |
| <b>Bifă</b>             | Eticheta, starea (bifat / nebifat), dacă bifarea e obligatorie      |
| <b>Listă de opțiuni</b> | Eticheta, opțiunile posibile, opțiunea aleasă                       |

Fiecare câmp știe singur dacă este **completat corect**; un formular este valid dacă toate câmpurile lui sunt valide.

**Manipulare** înseamnă:

- **Verificarea** formularului — se afișează, pe rând, ce câmp nu e completat corect și de ce;
- **Golirea** formularului — fiecare câmp revine la starea necompletată, dar își păstrează regulile;
- **Duplicarea** formularului ca șablon gol, pentru următorul înscris.

**Cerințe:** definește toate clasele necesare și scrie un program care completează greșit cel puțin două câmpuri, verifică formularul, îl corectează și îl verifică din nou. Metoda care verifică formularul **nu are voie să conțină** `instanceof`.